Design training data for AI workloads on Azure

When you design data for AI functionality in applications, consider both non-functional requirements, such as operability, cost, and security, and functional requirements that are related to data ingestion, preparation, and validation.

Data design and application design can't be decoupled. Application design requires that you understand use cases, query patterns, and freshness requirements. To address business requirements that drive the need for using AI, the application might need output from discriminative models, generative models, or a combination of model types.

To produce meaningful results, AI models need to be trained. Model training involves teaching a model to classify or predict new or unseen situations. The training data must be tailored to the specific problem and workload context.

Supervised training involves providing the model with labeled samples. This type of training is useful when the desired outcome is clear. In contrast, unsupervised learning allows the model to identify patterns and relationships within the data without guidance on the expected output. During training, the algorithm type and its parameters are adjusted to control how the model learns. The approach varies depending on the type of model, which can include neural networks, decision trees, and others.

For example, image detection models are typically trained on tasks like object detection, facial recognition, or scene understanding. They learn from annotated images to identify specific objects or features. Other common examples include fraud detection algorithms and price-point prediction models. These models learn from historical financial data to make informed decisions.

This article focuses primarily on the preceding use case, where models are trained *before* they can give meaningful input to the application. The article includes guidance on data collection, processing, storing, testing, and maintenance. Data design for exploratory data science or business intelligence via AI isn't covered. The goal is to support training needs through strategies that are aligned with workload requirements by providing recommendations on the training data pipeline of an AI workload.

For information about data design for AI models that require context *during* inferencing, see Grounding data design.

> **Important**
>
> Expect data design to be an iterative process that's based on statistical experimentation. To reach an acceptable quality level, adjust training data, its processing, model feature development, and model hyperparameters (when possible). This experimentation loop typically happens both during initial model training and during ongoing refinement efforts to address data and model drift over the lifespan of the feature in the workload.

Recommendations

Here's the summary of recommendations provided in this article.

Expand table

Recommendation	Description
Select data sources based on workload requirements.	Factor in available resources and whether the data source can help you reach the acceptable data quality for model training. Cover both positive and negative examples. Combine diverse data types to achieve adequate completeness for analysis and modeling. Consider techniques like Synthetic Minority Oversampling Technique (SMOTE) for data scarcity or imbalance. ▪ Data ingestion and analysis
Conduct data analysis on the collected data early.	Perform analysis processes, such as Exploratory Data Analysis (EDA), offline. Consider the costs and security implications. For small datasets without resource constraints, you can consider performing analysis at the source. ▪ Data collection store
Maintain data segmentation, if business and technical requirements call for it.	If you use data sources that have distinct security requirements, create separate pipelines for each model. Establish access controls to limit interaction with specific data subsets. ▪ Data segmentation
Preprocess data to make it meaningful against training goals.	Refine the quality of ingested data by filtering noise, rescoping the data, addressing duplicates, and standardizing diverse formats. ▪ Data preprocessing
Avoid training on stale data.	Monitor for data drift and concept drift as part of your inner and outer operational loops to maintain the accuracy and reliability of models over time. Regularly update training data with new observations. Define conditions that trigger model retraining and determine update frequency.

Recommendation	Description
	▪ Data maintenance

Types of data

To build predictive power in models, you need to collect data, process it, and feed it to the model. This process is usually conceptualized as a pipeline that's broken into stages. Each stage of the pipeline might deal with the same data set, but it might serve different purposes. Typically, you handle data of these types:

- *Source data* is point-in-time observation data. It can also be data that can be labeled to serve as a potential input to the data pipeline.

  This data is usually obtained from production or from an external source. These data sources can be in storage accounts, databases, APIs, or other sources. The data can be in various data formats, like OLTP databases, unstructured documents, or log files. This data serves as a potential input to the data pipeline.

- *Training data* is a subset of source data that's used for providing samples to the model. The samples are descriptive precalculated data that help the model learn patterns and relationships. Without this data, the model can't generate relevant output.

- *Evaluation data* is a subset of the source data that's used to monitor and validate the performance of a machine learning model during training. It's distinct from training and test data and is used to periodically evaluate the model's performance during the training phase and guide hyperparameter tuning. For more information, see Model evaluation.

- *Testing data* is used to validate the predictive power of a trained model. This data is sampled from source data that wasn't used for training. It contains observations from production so that the testing process is conclusive. From a data design perspective, you need to store this data. For information about testing models, see the Testing design area.

Data can be structured (databases, spreadsheets), semi-structured (JSON, XML), or unstructured (text documents, images). *Multi-modal data* combines different data types like text, images, audio, or video within the same training dataset. When working with multi-modal data, preserve cross-modal relationships and maintain semantic connections between different data types throughout processing and feature engineering. Ensure synchronized data processing so that related information across different modalities remains aligned during preprocessing and training.

In some cases, information that's provided by users during interactions with the application can eventually become source data. In general, we recommend that user input used this manner is of high quality. Otherwise, the need to continuously handle quality issues downstream can become problematic. Guidance about handling user data isn't covered in this article.

Data ingestion and analysis

Training data is collected within a predetermined window that has sufficient representations for training the type of model that you select. For example, when you train a binary classification model, training data must include representations of what is the case (positive examples) and what isn't the case (negative examples). For training data to be meaningful, conduct EDA early during feature design.

EDA helps analyze source data to identify characteristics, relationships, patterns, and quality issues. You can conduct EDA directly at the source data store or replicate data into centralized stores, like a data lake or data warehouse. The outcome of the process is to inform data collection and processing for effective model training.

> **Note**
>
> Although EDA is a pre-production process, it uses data that's sourced from production. Apply the same level of control to this process as you would for production.

Following are some considerations for collecting data in preparation for model training.

Data sources

Data can be collected from these sources:

- **Proprietary data** is created or owned by the organization. It's not intended for public consumption. It serves internal purposes.
- **Public sources** are accessible to anyone. These sources include websites, research papers, and publicly shared databases. It might be specific to a niche area. For example, content from Wikipedia and PubMed are considered publicly accessible.
- **User-generated data** includes information captured from user interactions, expert feedback, and collaborative workflows. This proprietary data supports continuous learning architectures where are updated incrementally as new information becomes available.

Design your data pipeline to accommodate feedback loops while maintaining data quality standards and preventing degradation from low-quality inputs.

Your choice of data sources depends on workload requirements, available resources, and the quality of the data that's acceptable for training the model. Imbalanced datasets can lead to biased models, so you need to design data collection to get sufficient samples of representative data. You might need to oversample minority data or undersample majority data. If the data is scarce or imbalanced, consider techniques like SMOTE and Synthetic data generation.

Data collection store

There are two main options for collecting source data:

- Querying the data at the data source
- Copying the data to a localized data store and then querying that store

The choice depends on workload requirements and the volume of data. If you have a relatively small amount of data, the source system might handle your raw queries directly. However, the common practice is to query and analyze from the localized store.

> **Tradeoff.** Although localized data stores might facilitate analysis and the training process, you also need to balance costs, security, and model requirements.
>
> Duplicating data incurs storage and compute costs. Maintaining a separate copy necessitates additional resources. Local copies might contain sensitive information. If it does, you need to protect the data by using regular security measures.
>
> If you use production data for training data, it must be subject to all the original data classification constraints of that data.

Data can be provided to the training process (push mode) or the process itself can query the data source (pull mode). The choice depends on ownership, efficiency, and resource constraints.

When data is pushed to the workload, it's the responsibility of the data source owner to provide fresh data. The workload owner provides a suitable location in their localized data store to store the data. This approach applies to proprietary data that's owned by the organization, not to public sources.

There are two approaches that you can use for pulling data. In one approach, the workload queries against the data store, retrieves necessary data, and places it in the localized store.

Another way is to perform real-time queries in memory. The decision depends on data volume and available compute resources. For smaller datasets, in-memory retrieval might be sufficient for model training.

Regardless of whether you use push or pull mode, avoid training models on stale data. The frequency of data updates should align with workload requirements.

Data segmentation

Workload-specific requirements might necessitate data segmentation. Here are some potential use cases:

- **Security requirements** often drive segmentation decisions. For instance, regulatory constraints might prevent exporting data across geopolitical regions. If your application design allows the use of separate models, data design incorporates separate data pipelines for each model.

  However, if a single model is used, segmented data sources feed into that model. You need to train the model on data from both geographies, which potentially adds complexity.

  Whether the application is using a single model or multiple models, preserve security measures on each data segment so that it's protected with the same level of rigor as data at the origin.

- **Data freshness rate** can be a factor for separating data. Data from different sources might refresh at varying time intervals. If the data changes, retraining becomes necessary. Segmentation enables granular control of the data lifecycle. Consider using separate tables or pipelines for different data segments.

Regardless of the use case, when data is segmented, access controls are key. Data professionals, like data engineers and data scientists, explore available source data to understand patterns and relationships. Their insights contribute to training models that predict outcomes. Establish access controls to ensure that only authorized users can interact with specific data subsets. Apply least privilege on data that's considered to be relevant. Collaborate with data owners to set up appropriate permissions.

Data preprocessing

In a real-world scenario, source data isn't simply stored for AI scenarios. There's an intermediate process that prepares data for training. During this stage, data is stripped of noise, making it

useful for consumption. When dealing with source data, data scientists engage in a process of exploration, experimentation, and decision making. Their primary goal is to identify and extract parts of the source data that holds predictive power.

The preprocessing logic depends on the problem, data type, and desired outcomes. Following are some common techniques for preprocessing. This list isn't exhaustive. The actual criteria for your workload will be driven by business requirements.

- **Quality**. Preprocessing can help you ensure that training data is stripped of noise. The goal is to ensure that every row in your training data represents a clear observation or a good example that's relevant to your use case and to eliminate observations that lack quality or predictive power. For example, if you collate product reviews, you might choose to eliminate data that's too short. You need to discover what data quality produces meaningful predictive results.

- **Rescoping**. Source data fields that are too specific can restrict predictive powers. For example, consider an address field. Broadening the scope from full address (house number and street name) to a higher level, like city, state, or country/region, might be more relevant.

- **Deduplication**. Eliminating redundancy can ensure that your training data remains accurate and representative. In certain cases, the frequency with which an observation is made isn't relevant. For example, when you scan logs, if a log entry appears 1,000 times, that indicates its frequency. It doesn't necessarily imply that it's a more serious error than a log that occurred only once. This type of redundancy can introduce noise.

- **Sensitive data handling**. Eliminate personal data unless it's absolutely vital to the model's predictive power in a way that can't be achieved through anonymization. Training data should be effective without compromising privacy. If the data provides value, you need to be aware of the ethical considerations of handling sensitive data. For more information, see Responsible AI.

- **Standardized transformation**. Domain experts consider the preceding techniques to be a core part of feature engineering. Broad scope and diverse source data eventually need to merge into feature stores where features are organized (for example, into feature tables) for the explicit purpose of training models. After you select predictive data for training, transform the data to a standardized format. Standardization also ensures compatibility with the training model.

Converting images into text representations is a form of transformation. For instance, you might convert scanned documents or images to machine-readable text.

To ensure compatibility with models, you might need to adjust orientations or aspect ratios of images to match the model's expectations.

> **Note**
>
> Mixing large amounts of structured and unstructured data can increase processing time. Workload teams should measure the impact of processing diverse formats. As the window between retraining efforts becomes shorter, the amount of time spent on preprocessing becomes more critical.

Feature store design

Feature stores serve as centralized repositories for storing, managing, and serving engineered features for machine learning models. They provide consistency across training and inference phases while enabling feature reuse across multiple models and teams. Organizations can implement multiple approaches to implement the feature stores:

- **Centralized feature stores** provide a single source of truth for all features across an organization. All teams contribute to and consume from a shared feature repository. This approach promotes consistency and reduces duplication but requires strong governance and standardization practices.
- **Distributed feature stores** allow teams to maintain their own feature repositories while providing mechanisms for controlled sharing. This pattern offers greater autonomy for individual teams but requires careful coordination to prevent feature inconsistencies across models.
- **Hybrid approaches** combine centralized governance with distributed implementation. Common features are managed centrally while domain-specific features remain with individual teams.

Workload teams in organizations with strong data governance teams and standardized ML practices will likely follow the centralized approach, or a hybrid model if they need to comply with the organization standards while still having some autonomy. Distributed patterns suit organizations where teams have distinct data requirements and operate with high autonomy.

Design considerations

- Design clear separation between feature definition, storage, and serving layers
- Organize features into logical groupings that align with business domains and model requirements
- Establish consistent naming conventions and schema evolution practices
- Make features immutable once published, with new versions created for changes
- Integrate with data preprocessing, model training, and model serving infrastructure
- Implement automated checks for data drift, feature distribution changes, and quality degradation

Foundation model fine-tuning

Training data for foundation model fine-tuning follows different patterns than traditional model training from scratch. Instead of training entire models, you're adapting pre-trained capabilities to specific domains or tasks.

Domain-specific fine-tuning requires high-quality, task-relevant training examples that represent the specific patterns you want the model to learn. The volume of training data is typically smaller than traditional training scenarios, but the quality and representativeness become more critical.

For example, fine-tuning a general language model for medical documentation requires training examples that accurately represent medical terminology, clinical reasoning patterns, and documentation standards specific to your organization or domain.

Task-specific adaptation focuses training data on particular capabilities or output formats. This approach might involve training the model to follow specific instructions, produce structured outputs, or integrate with particular workflows.

Consider how your training data pipeline can efficiently provide curated, high-quality examples for fine-tuning while maintaining traceability and version control for different model variants.

Data retention

After you train a model, evaluate whether to delete the data used for training and rebuild the model for the next training window.

If the data remains relatively unchanged, retraining might not be necessary, unless model drift occurs. If the accuracy of prediction decreases, you need to retrain the model. You can choose to

ingest the data again, preprocess, and build the model. That course of action is best if there's a significant delta in data since the last training window. If there's large volume of data and it hasn't changed much, you might not need to preprocess and rebuild the model. In that case, retain data, do in-place updates, and retrain the model. Decide how long you want to retain training data.

In general, delete data from feature stores to reduce clutter and storage costs for features that have poor performance and that are no longer relevant to current or future models. If you retain data, expect to manage costs and address security issues, which are typical concerns of data duplication.

Lineage tracking

Data lineage refers to tracking the path of data from its source to its use in model training. Keeping track of data lineage is essential for explainability. Although users might not need detailed information about data origins, that information is crucial for internal data governance teams. Lineage metadata ensures transparency and accountability, even if it's not directly used by the model. This is useful for debugging purposes. It also helps you determine whether biases are introduced during data preprocessing.

Use platform features for lineage tracking when you can. For example, Azure Machine Learning is integrated in Microsoft Purview. This integration gives you access to features for data discovery, lineage tracking, and governance as part of the MLOps lifecycle.

Data maintenance

All models can become stale over time, which causes a model's predictive power or relevance to decay. Several external changes can cause decay, including shift in user behavior, market dynamics, or other factors. Models trained some time ago might be less relevant because of changing circumstances. To make predictions with better fidelity, you need recent data.

- **Adopting newer models**. To ensure relevance, you need an operational loop that continuously evaluates model performance and considers newer models, which keeps the data pipeline minimally disruptive. You can alternatively prepare for a bigger change that involves redesigning the data lifecycle and pipeline.

  When you choose a new model, you don't necessarily need to start with a new data set. The existing observations used for training might remain valuable even during a model switch. Although new models might reveal narrower scenarios, the fundamental process remains

similar. Data management approaches like feature stores and data meshes can streamline the adoption of new machine learning models.

- **Trigger-based vs. routine operations**. Consider whether model retraining should be triggered by specific events or conditions. For example, the availability of new, more relevant data or a drop in relevancy below an established baseline might trigger retraining. The advantages of this approach are responsiveness and timely updates.

  Maintenance can also be scheduled at regular fixed intervals, like daily or weekly. For fail-proof operations, consider both approaches.

- **Data removal**. Remove data that's no longer being used for training to optimize resource use and minimize the risk of using outdated or irrelevant data for model training.

  The right to be forgotten refers to an individual's right to have their personal data removed from online platforms or databases. Be sure to have policies in place to remove personal data that's used for training.

- **Data retention**. In some situations, you need to rebuild an existing model. For example, for disaster recovery, a model should be regenerated exactly as it was before the catastrophic event. We recommend that you have a secondary data pipeline that follows the workload requirements of the primary pipeline, like addressing model decay, regular updates via trigger-based or routine operations, and other maintenance tasks.

> **Tradeoff.** Data maintenance is expensive. It involves copying data, building redundant pipelines, and running routine processes. Keep in mind that regular training might not improve answer quality. It only provides assurance against staleness. Evaluate the importance of data changes as a signal to determine the frequency of updates.

Make sure that data maintenance is done as part of model operations. You should establish processes to handle changes via automation as much as possible and use the right set of tools. For more information, see MLOps and GenAIOps for AI workloads on Azure.

Next steps

Design area: Grounding data design

Last updated on 10/30/2025